

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	Total Marks:	1 * 100 = 100 (1 Question1)
CO2 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	A. Mounish	Reg. No.:	192472273
Section / Batch:	CSE AI	Date:	03/08/2026

Instructions to Students

- Answer 1 question. Total: 100 Marks.
- Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
- Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
- Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
- Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
- Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
- Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
- Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
- Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application. **Submission Requirements Students should submit:**

- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct

*I **A. Mounish / 192472273** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

SECTION A — FUNCTION CALLING & STRUCTURED OUTPUT TASK

30. Secure Function Calling

A banking assistant can check account details and initiate money transfers.

Task: Design a secure function-calling workflow that requires confirmation before executing a transfer.

Identify the parameters, validation rules, and structured response required.

Aim

Design a secure function-calling workflow for a banking assistant that checks account details and transfers money only after user confirmation.

Workflow

1. User requests account balance or money transfer.
2. Assistant identifies the required function.
3. Assistant collects transfer details (receiver account, amount, purpose).
4. Validate all input values.
5. Display transfer summary and ask for user confirmation.
6. Execute transfer only if the user replies "Confirm".
7. Return success or failure response.

Function Parameters

Parameter	Type	Description
sender_account	String	Sender account number
receiver_account	String	Receiver account number
amount	Float	Amount to transfer
purpose	String	Reason for transfer
confirmation	Boolean	User confirmation (True/False)

Validation Rules

- Account numbers must exist.
- Sender and receiver accounts cannot be the same.
- Transfer amount must be greater than 0.
- Sender must have sufficient balance.
- User confirmation must be True before transfer.
- Reject invalid or missing inputs.

Python Code:

```
# Secure Function Calling - Banking Assistant
```

```
# Sample account database
```

```
accounts = {  
    "1001": 10000,  
    "1002": 5000,  
    "1003": 8000  
}
```

```
# Function to check balance
```

```
def check_balance(account):  
    if account in accounts:  
        return f"Account Balance: ₹{accounts[account]}"  
    else:  
        return "Invalid Account Number"
```

```
# Function to transfer money
```

```
def transfer_money(sender, receiver, amount):  
    # Validation  
    if sender not in accounts:  
        return {"status": "failed", "reason": "Sender account not found"}  
  
    if receiver not in accounts:  
        return {"status": "failed", "reason": "Receiver account not found"}  
  
    if sender == receiver:  
        return {"status": "failed", "reason": "Sender and receiver cannot be the same"}  
  
    if amount <= 0:  
        return {"status": "failed", "reason": "Invalid transfer amount"}  
  
    if accounts[sender] < amount:  
        return {"status": "failed", "reason": "Insufficient balance"}  
  
    # Confirmation  
    print("\nTransfer Summary")  
    print("-----")  
    print("From :", sender)  
    print("To   :", receiver)  
    print("Amount : ₹", amount)  
  
    confirm = input("Type 'CONFIRM' to proceed: ")  
  
    if confirm.upper() != "CONFIRM":  
        return {  
            "status": "cancelled",
```

```

        "message": "Transfer cancelled by user"
    }

# Execute transfer
accounts[sender] -= amount
accounts[receiver] += amount

return {
    "status": "success",
    "message": "Money transferred successfully",
    "sender_balance": accounts[sender],
    "receiver_balance": accounts[receiver]
}

# Main Program
print("===== Banking Assistant =====")
print("1. Check Balance")
print("2. Transfer Money")

choice = input("Enter your choice: ")

if choice == "1":
    acc = input("Enter Account Number: ")
    print(check_balance(acc))

elif choice == "2":
    sender = input("Sender Account: ")
    receiver = input("Receiver Account: ")
    amount = float(input("Amount: ₹"))
    result = transfer_money(sender, receiver, amount)
    print(result)

else:
    print("Invalid Choice")

```

Output

```

===== Banking Assistant =====
1. Check Balance
2. Transfer Money
Enter your choice: 2
Sender Account: 1001
Receiver Account: 1002
Amount: ₹3000

```

Transfer Summary

From : 1001
To : 1002
Amount : ₹ 3000.0
Type 'CONFIRM' to proceed: CONFIRM
{'status': 'success', 'message': 'Money transferred successfully', 'sender_balance': 7000.0, 'receiver_balance': 8000.0}

Results
A secure banking workflow is designed where the assistant validates all inputs and executes the money transfer only after explicit user confirmation, ensuring safe and reliable transactions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						

Overall Total (100)		
----------------------------	--	--

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____